HDOC Day-55

Sliding Window Maximum Using Pointers and Nested Loops

Problem Statement

Read the size and elements of a variable-length integer array arr[] and an integer k. Pass the array by address to window_maximum(). For every contiguous window of size k, print its maximum from left to right separated by spaces. If k is not in the range 1 to size, print -1. Use nested loops.

Algorithm

1. Start.
2. Read positive array size n and validate it.
3. Declare the variable-length integer array arr[n] and read n integers.
4. Read integer k.
5. Call window_maximum(arr, n, k) by passing the array address.
6. If k <= 0 or k > n, print -1 and return.
7. For each starting index i from 0 to n-k, set maximum = arr[i].
8. Using an inner loop, compare elements arr[i+1] through arr[i+k-1] and update maximum.
9. Print each window maximum separated by a space.
10. Stop.

Test Case Table

TC	Size	Array elements	k	Expected output	Purpose
1	8	1 3 -1 -3 5 3 6 7	3	3 3 5 5 6 7	Professor example 1
2	5	1 2 3 4 5	2	2 3 4 5	Professor example 2
3	3	1 2 3	4	-1	k greater than size
4	4	4 2 7 1	1	4 2 7 1	Smallest valid k
5	4	4 2 7 1	4	7	k equals size
6	5	-5 -2 -8 -1 -6	2	-2 -2 -1 -1	All negative elements
7	5	2 2 2 2 2	3	2 2 2	Duplicate values
8	3	3 1 2	0	-1	Zero k
9	3	3 1 2	-2	-1	Negative k
10	1	9	1	9	Single-element array

C Program

```
#include <stdio.h>

void window_maximum(const int *arr, int size, int k)
{
    int i, j, maximum;

    if (k <= 0 || k > size)
    {
        printf("-1\n");
        return;
    }

    printf("Window maximums: ");
    for (i = 0; i <= size - k; i++)
    {
```

```
        maximum = *(arr + i);
        for (j = i + 1; j < i + k; j++)
        {
            if (*(arr + j) > maximum)
                maximum = *(arr + j);
        }
        if (i > 0)
            printf(" ");
        printf("%d", maximum);
    }
    printf("\n");
}

int main(void)
{
    int size, k, i;

    printf("Enter array size: ");
    if (scanf("%d", &size) != 1 || size <= 0 || size > 10000)
    {
        printf("Invalid size. Enter a value from 1 to 10000.\n");
        return 1;
    }

    int arr[size];
    printf("Enter %d array elements:\n", size);
    for (i = 0; i < size; i++)
    {
        if (scanf("%d", arr + i) != 1)
        {
            printf("Invalid array element.\n");
            return 1;
        }
    }

    printf("Enter k: ");
    if (scanf("%d", &k) != 1)
    {
        printf("Invalid k.\n");
        return 1;
    }

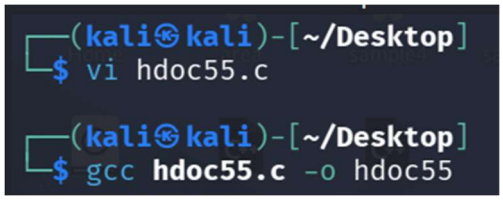
    window_maximum(arr, size, k);
    return 0;
}
```

Compilation and Execution Commands

```
gcc hdoc55.c -o hdoc55
```

```
./hdoc55
```

Compilation and Execution



Test Case 1

Input: size = 8; elements = 1 3 -1 -3 5 3 6 7; k = 3

Expected output: 3 3 5 5 6 7

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc55
Enter array size: 8
Enter 8 array elements:
1 3 -1 -3 5 3 6 7
Enter k: 3
Window maximums: 3 3 5 5 6 7
```

Test Case 2

Input: size = 5; elements = 1 2 3 4 5; k = 2

Expected output: 2 3 4 5

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc55
Enter array size: 5
Enter 5 array elements:
1 2 3 4 5
Enter k: 2
Window maximums: 2 3 4 5
```

Test Case 3

Input: size = 3; elements = 1 2 3; k = 4

Expected output: -1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc55
Enter array size: 3
Enter 3 array elements:
1 2 3
Enter k: 4
-1
```

Test Case 4

Input: size = 4; elements = 4 2 7 1; k = 1

Expected output: 4 2 7 1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc55
Enter array size: 4
Enter 4 array elements:
4 2 7 1
Enter k: 1
Window maximums: 4 2 7 1
```

Test Case 5

Input: size = 4; elements = 4 2 7 1; k = 4

Expected output: 7

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc55
Enter array size: 4
Enter 4 array elements:
4 2 7 1
Enter k: 4
Window maximums: 7
```

Test Case 6

Input: size = 5; elements = -5 -2 -8 -1 -6; k = 2

Expected output: -2 -2 -1 -1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc55
Enter array size: 5
Enter 5 array elements:
-5 -2 -8 -1 -6
Enter k: 2
Window maximums: -2 -2 -1 -1
```

Test Case 7

Input: size = 5; elements = 2 2 2 2 2; k = 3

Expected output: 2 2 2

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc55
Enter array size: 5
Enter 5 array elements:
2 2 2 2 2
Enter k: 3
Window maximums: 2 2 2
```

Test Case 8

Input: size = 3; elements = 3 1 2; k = 0

Expected output: -1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc55
Enter array size: 3
Enter 3 array elements:
3 1 2
Enter k: 0
-1
```

Test Case 9

Input: size = 3; elements = 3 1 2; k = -2

Expected output: -1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc55
Enter array size: 3
Enter 3 array elements:
3 1 2
Enter k: -2
-1
```

Test Case 10

Input: size = 1; elements = 9; k = 1

Expected output: 9

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc55
Enter array size: 1
Enter 1 array elements:
9
Enter k: 1
Window maximums: 9
```